

1 Il processo

In un moderno sistema sono caricati in memoria diversi programmi che sono eseguiti contemporaneamente. Questo avviene sia per i computer gestiti da sistemi operativi che supportano la multiutenza, sia nei personal computer, dove un singolo utente può avere molti programmi contemporaneamente attivi. I programmi in esecuzione devono essere descritti e la loro evoluzione deve essere controllata. Lo strumento concettuale usato per descrivere un programma in esecuzione è il **processo**.

Per descrivere i programmi in esecuzione si usano diversi termini quali *job*, *task* degli utenti, *task* del sistema operativo. Tutti questi termini possono essere intesi come sinonimi di *processo*. Un intero sistema è descritto come un insieme di processi che possono essere eseguiti in parallelo sui processori e il sistema operativo ne alterna l'esecuzione.

L'alternarsi delle esecuzioni avviene o per effetto di azioni generate dagli stessi processi, come nel caso di un programma in esecuzione che si ferma in attesa di ricevere dati da un'unità periferica, oppure a causa di azioni esterne al processo, come nel caso di un segnale inviato dal timer per interrompere un processo una volta scaduto il quanto di tempo assegnato. Un processo che si è fermato, perché aspetta i dati da un'unità periferica, riprenderà l'esecuzione dopo che i dati sono stati trasferiti in memoria. Al termine del trasferimento la periferica informa, con un segnale, che i dati sono pronti.

Un ruolo importante nell'alternarsi delle esecuzioni è la capacità di interrompere un programma attivo, in modo che la sua esecuzione possa essere ripresa al punto dell'interruzione. Il concetto di *interruzione* e i meccanismi che la governano sono molto importanti per l'intero funzionamento del sistema.

2 Le interruzioni

Nel precedente capitolo si è visto che il processore riesce a eseguire molti programmi contemporaneamente alternandone l'esecuzione nel tempo. L'alternarsi delle esecuzioni avviene secondo diverse modalità e per differenti cause:

- il sistema operativo interrompe l'esecuzione di un programma a bassa priorità per assegnare il processore a un programma a priorità più elevata;
- nel caso dei sistemi *time sharing*, allo scadere di una quantità prefissata di tempo, il programma in esecuzione è interrotto per cedere il controllo della CPU (processore) ad un altro programma;
- un programma in esecuzione che deve leggere dei dati da una periferica richiede l'intervento del sistema operativo che avvia l'operazione di lettura; mentre il programma che ha fatto la richiesta aspetta l'arrivo dei dati, il sistema operativo manda in esecuzione un altro programma;
- un evento esterno al sistema causa l'interruzione del programma in esecuzione, permettendo al sistema operativo di intervenire per attuare le azioni necessarie. Per esempio, un sensore collegato alla porta di una camera d'albergo rileva l'apertura della stanza, il televisore della camera si deve accendere e sullo schermo appare un messaggio personalizzato di benvenuto al nuovo ospite.

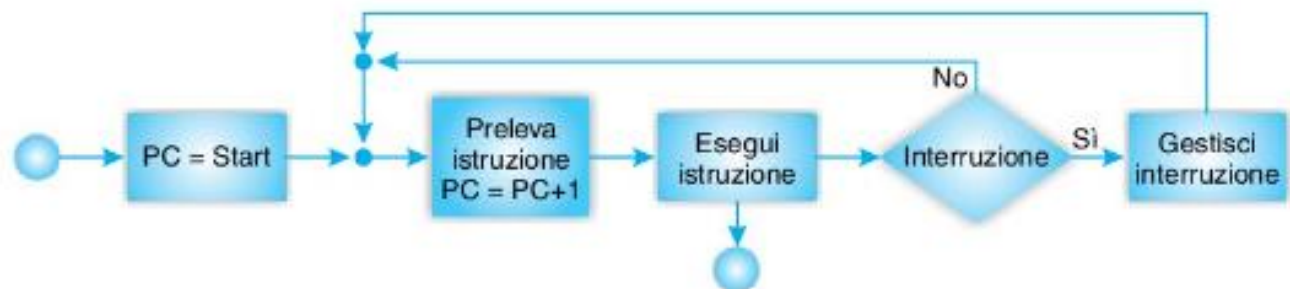
Perché questo avvenga bisogna che l'esecuzione di un programma possa essere interrotta. L'esecuzione delle istruzioni macchina può essere descritta con il seguente schema:



Il processore, dopo avere eseguito un'istruzione, preleva la prossima istruzione, la esegue e poi ripete il ciclo. Lo schema mostra che un programma, una volta iniziato, proseguirà nell'esecuzione sino al suo naturale completamento. In realtà lo schema è corretto solo se le interruzioni sono *disabilitate*. Quando le interruzioni sono *abilitate*, il ciclo di esecuzione delle istruzioni prevede che, dopo avere eseguito un'istruzione, il processore controlli se ci sono segnali di interruzione pendenti. In caso affermativo, il processore:

- prima gestisce l'interruzione trasferendo il controllo a una routine di **gestione dell'interruzione**;
- successivamente, passa a eseguire la prossima istruzione. È possibile naturalmente che, a causa della gestione dell'interruzione, la prossima istruzione eseguita sia quella di un altro programma.

Lo schema precedente deve essere quindi modificato in questo modo:



L'abilitazione o meno delle interruzioni è gestita dal sistema operativo che modifica il valore di un apposito *flag* memorizzato nella **PSW** (*Program Status Word*). L'istruzione che permette di cambiare il valore del bit di abilitazione delle interruzioni è, naturalmente, un'istruzione privilegiata e può essere eseguita solo quando il processore opera in modalità *supervisore*.

Il concetto di interruzione fa parte della vita di tutti i giorni. Per esempio, mentre stiamo guardando un programma televisivo, può accadere di essere interrotti da una telefonata. Si osservi anche come il meccanismo dell'interruzione ci permetta di gestire con efficienza i fatti della nostra vita. Se aspettiamo la visita di una persona, possiamo continuare nelle nostre normali attività anche durante l'attesa, evitando di passare il tempo sulla porta di casa ad aspettare l'arrivo del visitatore.

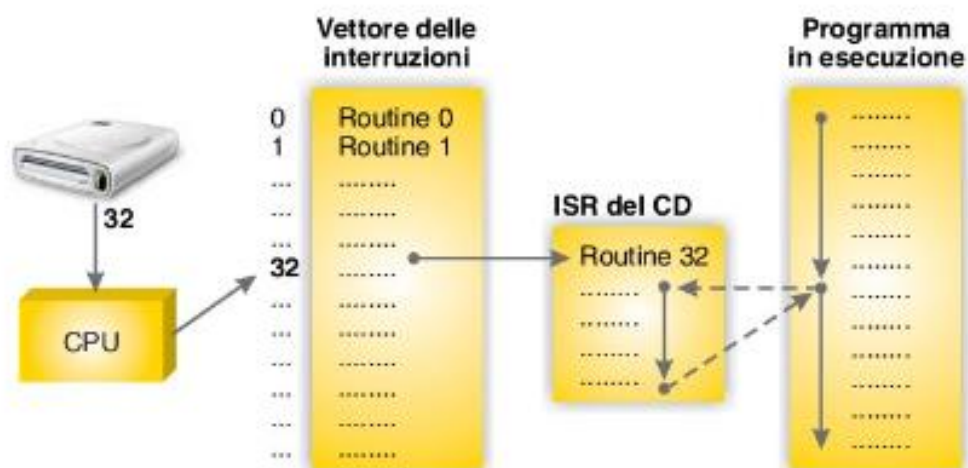
I computer gestiscono l'interruzione in modo analogo alle persone quando devono interrompere la propria attività per rispondere a un segnale di allerta. Per esempio, una persona che sente il suono di un campanello mentre sta scrivendo una relazione segue un comportamento come il seguente:

1. salva il proprio lavoro per poterlo riprendere esattamente da dove lo ha interrotto;
2. in base al suono del campanello capisce se deve rispondere al telefono o aprire la porta a un visitatore oppure controllare lo stato di cottura di una pizza messa nel forno;
3. esegue le azioni necessarie per gestire l'interruzione: in base al tipo di segnale risponde al telefono, oppure apre la porta di casa per ricevere un visitatore, oppure estrae la teglia dal forno;
4. riprende il lavoro interrotto.

In analogia al comportamento umano un programma per la gestione dell'interruzione, indicato con la sigla **ISR** (*Interrupt Service Routine*), deve:

1. salvare lo stato del programma in esecuzione per farlo ripartire al termine dell'interruzione;
2. identificare chi ha inviato la richiesta di interruzione e quindi il tipo di servizio richiesto;
3. trasferire il controllo alla routine specializzata nella gestione del servizio richiesto.

Le interruzioni possibili sono in numero predefinito. Per velocizzare le operazioni di gestione dell'interruzione si usa un **vettore delle interruzioni** (*interrupt vector*), cioè un array di puntatori a differenti routine di gestione dell'interruzione. Il vettore delle interruzioni consiste in un insieme di celle di memoria contigue, ognuna delle quali contiene l'*entry point* di una routine specializzata in un servizio. Ad ogni dispositivo o evento che genera interruzioni è associato un valore numerico che viene usato come indice per accedere al vettore delle interruzioni e far partire la routine in grado di servire le interruzioni di quel dispositivo (*ISR del dispositivo*). Questa tecnica è usata da molti sistemi operativi tra i quali Dos e Unix/Linux.



Si consideri per esempio il caso di un CD-ROM, identificato come dispositivo 32, che invia un segnale di interruzione per informare di essere pronto a trasmettere un blocco di dati:

1. Il CD-ROM memorizza il blocco di dati richiesto nel buffer del controllore del dispositivo e invia un segnale di interruzione al processore.
2. Il processore completa l'esecuzione dell'istruzione corrente e poi esamina il registro delle interruzioni per identificare il dispositivo che ha causato l'interruzione. Invia un segnale di risposta al dispositivo per informarlo che l'interruzione è stata riconosciuta.
3. Il processore usa il numero del dispositivo per accedere alla posizione 32 del vettore delle interruzioni. In quella cella trova il valore dell'*entry point* della routine di gestione dell'interruzione del CD (*ISR del CD*).
4. Il processore, prima di trasferire il controllo alla *Routine 32*, deve salvare lo stato del programma in esecuzione: trasferisce in locazioni prefissate della memoria centrale il valore del *Program Counter*, dei registri utente e delle informazioni contenute nella *PSW*. In questo modo vengono salvati l'indirizzo della prossima istruzione da eseguire, i risultati intermedi salvati nei registri, i bit di stato che sono stati settati dalle precedenti operazioni e che possono influenzare le istruzioni successive.
5. Il processore carica nel *Program Counter* l'indirizzo che ha prelevato dal vettore delle interruzioni avviando così l'esecuzione della *Routine 32*.
6. Il processore esegue la *ISR del CD* che contiene le istruzioni necessarie per elaborare l'interruzione. Nel caso specifico il processore trasferisce i dati dal controllore del CD alla memoria centrale e poi avverte il controllore di avere svuotato il buffer, che potrà quindi essere riempito con altri dati.
7. Completata l'elaborazione dell'*interrupt*, il programma interrotto deve ripartire: il processore ripristina, nell'ordine, lo stato della *PSW*, dei registri utente e del *Program Counter*, prelevando dalla memoria i valori salvati nel precedente punto 4. In questo modo l'esecuzione del programma interrotto riparte dal punto di interruzione.

Il punto 7 è corretto solo se c'è un solo programma in esecuzione. In un sistema multiprogrammato bisogna tenere conto del fatto che, dopo avere ricevuto i dati inviati dal CD, il programma che li attende è pronto a ripartire. La decisione se far ripartire il programma appena interrotto, o quello che attendeva i dati dal CD, dipende dalle strategie del sistema operativo che, agendo in base alle priorità, potrebbe decidere di mandare in esecuzione non il programma interrotto, ma quello che attendeva i dati dal CD. Il punto 7 deve quindi essere formulato nel seguente modo più corretto:

Completata l'elaborazione dell'*interrupt* deve ripartire un programma precedentemente interrotto ripristinandone lo stato di esecuzione. Il sistema operativo decide, in base alle proprie politiche, quale programma far ripartire.

4 Stati di un processo e transizioni di stato

Per capire la vita di un processo, dalla nascita alla sua terminazione, occorre esaminare la sequenza di **stati** che assume e i relativi cambiamenti o **transizioni di stato**, identificando ogni volta l'entità che ha causato la transizione.

Facendo clic su un'icona o digitando un comando dal prompt dei comandi, si richiede di mandare in esecuzione un programma. Il sistema operativo crea la struttura dati che rappresenta il processo, ma non assegna ad esso alcuna risorsa: il processo si trova nello stato **new** (nuovo). Non appena possibile il sistema operativo assegna al processo le risorse richieste e lo colloca in memoria: si dice che il processo si trova nello stato **ready** (pronto). Un processo nello stato *ready* è, in pratica, un processo che può essere eseguito e attende l'assegnazione di un processore. Quando il sistema operativo decide di eseguire effettivamente il processo assegnando ad esso il processore, il processo passa nello stato **running** (esecuzione).

In un sistema con un solo processore ci può essere un solo processo *running*, ma ci possono essere molti processi *ready* in memoria. Il numero di processi contemporaneamente presenti in memoria definisce il *grado di multiprogrammazione* del sistema: all'aumentare del grado di multiprogrammazione, aumenta la percentuale di attività del processore e aumenta la probabilità di avere almeno un processo nello stato *ready*.

Un processo creato, messo in memoria e mandato in esecuzione, si trova nello stato *running*. Supponiamo che il processo stia girando in un ambiente *time sharing*. A questo punto l'evoluzione del processo può seguire tre strade diverse:

- Il processo completa la sua esecuzione e cede il controllo al sistema operativo che lo mette nello stato **terminated** (terminazione): il processo abbandona il sistema.
- Il quanto di tempo assegnato al processo scade e l'orologio di sistema invia un segnale di interruzione. La gestione dell'interruzione causa l'intervento del sistema operativo che mette il processo interrotto nello stato *ready* e, successivamente, seleziona uno dei processi *ready* per l'esecuzione.
- Il processo esegue un'operazione di Input/Output (I/O) e ne deve attendere il completamento. Esso richiede i servizi del sistema operativo specificando l'operazione da eseguire. Il sistema operativo attiva l'operazione di I/O, mette il processo nello stato **waiting** (attesa) e seleziona uno dei processi *ready* per l'esecuzione. Un processo nello stato *waiting* è quindi, in generale, un processo bloccato in attesa di qualche evento. Al completamento dell'operazione di I/O, il dispositivo interessato invia un segnale di interruzione che provoca l'intervento del sistema operativo. Il sistema operativo, poiché i dati richiesti sono ora disponibili, sposta il processo dallo stato *waiting* allo stato *ready*.



Riassumendo, le transizioni di stato di un processo sono:

- | | |
|-----------------------|--|
| Ammissione | è la transizione da <i>new</i> a <i>ready</i> ed è provocata da un utente (o da un altro processo) che avvia un processo. |
| Assegnazione | è la transizione da <i>ready</i> a <i>running</i> , consiste nell'assegnare un processore a un processo ed è eseguita dal sistema operativo seguendo le proprie politiche di schedulazione. |
| Interruzione | è la transizione da <i>running</i> a <i>ready</i> ed è causata da un'interruzione. Il processo potrebbe proseguire, ma viene interrotto da un evento esterno al processo: per esempio, un segnale inviato dall'orologio. |
| Attesa evento | è la transizione da <i>running</i> a <i>waiting</i> ed è causata dal processo stesso che, per eseguire un'operazione di I/O, chiama una routine di sistema che provoca l'intervento del sistema operativo. |
| Avviene evento | è la transizione da <i>waiting</i> a <i>ready</i> ed è causata da un evento esterno al processo, per esempio il completamento di un'operazione di Input/Output. |

Si può osservare che:

- Il sistema operativo è un programma sempre in esecuzione, nel senso che il codice del sistema operativo è sempre presente nella memoria centrale del computer pronto per l'esecuzione. La parte del sistema operativo sempre residente in memoria centrale si chiama **nucleo** (o *kernel*) del sistema operativo.
- Il sistema operativo è eseguito in *modalità riservata*; il processore, infatti, può funzionare in **modalità utente** e in **modalità riservata**: nella *modalità riservata* il processore è abilitato a eseguire tutte le istruzioni che compongono il set di istruzioni macchina, comprese le più delicate, e potenzialmente pericolose, per il funzionamento del computer; in *modalità utente* il processore può invece eseguire un insieme più ristretto di istruzioni macchina escludendo, per ragioni di sicurezza, alcune istruzioni la cui esecuzione è permessa solo al sistema operativo. Si dice anche che queste istruzioni sono permesse quando il processore è eseguito in modo **supervisore** o in *modalità kernel* (o anche in *modo di sistema*).

4

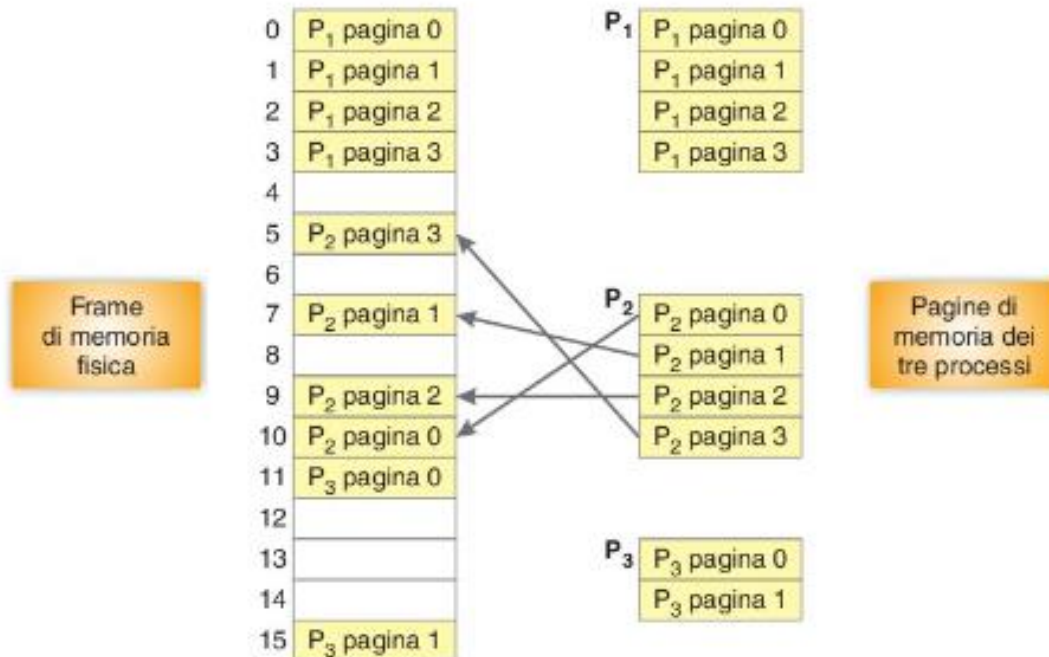
La paginazione

Le tecniche di allocazione della memoria sono dette di **allocazione contigua**, per indicare il fatto che, per assegnare la memoria a un processo, occorre trovare uno spazio di memoria libera abbastanza grande per contenerlo completamente. Purtroppo con le tecniche di allocazione contigua succede spesso di non riuscire ad allocare un processo in memoria perché, a causa della frammentazione, si formano molti spazi liberi di piccola dimensione che non sono in grado di accogliere alcun processo. Si consideri, per esempio, una memoria con un totale di 17 MB di memoria libera distribuita in 7 zone da 2 MB, 1 MB, 3 MB, 4 MB, 2 MB, 3 MB, 2 MB. Supponiamo che arrivi un nuovo processo da 5 MB. Per poterlo eseguire bisogna compattare la memoria generando un unico spazio libero da 17 MB: diversamente il processo deve aspettare che si liberino altre zone di memoria sino ad avere uno spazio in grado di accogliere il processo.

Per superare i limiti dell'allocazione contigua si devono utilizzare tecniche di allocazione che permettano di distribuire un processo in zone della memoria anche *non contigue*, lasciando alla MMU (*Memory Management Unit*) il compito di tradurre gli indirizzi *logici* generati dal processo nello spazio di memoria *fisica* a disposizione: questa idea è alla base della *paginazione* della memoria.

La **paginazione** è realizzata nel seguente modo: lo spazio di memoria (*memoria fisica*) è suddiviso in blocchi, aventi tutti la stessa dimensione, detti **frame** (cornici). Lo spazio di memoria dei processi (*memoria logica*) è suddiviso in blocchi di dimensione uguale a quella dei frame, detti **pagine**. La dimensione delle pagine di memoria dipende, naturalmente, dallo specifico sistema di gestione della memoria; un valore tipico della dimensione delle pagine è, per esempio, di 4 KB. L'allocazione della memoria a un processo viene fatta assegnando alle pagine del processo frame di memoria liberi non necessariamente contigui.

La figura seguente mostra la situazione di tre processi P_1 , P_2 e P_3 allocati in una memoria fisica di 16 frame numerati da 0 a 15. Le frecce che collegano le pagine del processo P_2 con i frame della memoria fisica visualizzano il **mapping** tra pagine e frame ed evidenziano la non contiguità delle pagine di P_2 in memoria. Anche P_3 non è allocato in pagine contigue: la pagina 0 di P_3 è collocata nel frame 11, mentre la pagina 1 è in memoria nel frame 15.



Per tener conto dell'allocazione delle pagine nei frame e dello stato della memoria servono opportune tabelle che descrivono il *mapping* tra pagine e frame. Di conseguenza, per ogni processo, occorre abbinare al numero di pagina logica il frame in cui essa è inserita.

Passiamo ora ad indirizzi in base 2 e supponiamo di avere indirizzi di memoria a 16 bit e una memoria suddivisa in 16 frame, aventi ciascuno 4 K celle di memoria: cioè la memoria è composta di $2^{16} = 64$ K parole ed è suddivisa in 16 frame da 4 K. Queste considerazioni valgono sia per gli indirizzi di memoria visti dal processo (*indirizzi logici*), che per gli indirizzi della memoria fisica (*indirizzi fisici*): lo spazio di memoria logica è composto da 16 pagine di 4 K e la memoria fisica è composta da 16 frame di 4 K. Osserviamo infine che la dimensione della pagina è una potenza di 2: $4\text{ K} = 2^2 \cdot 2^{10} = 2^{12}$.

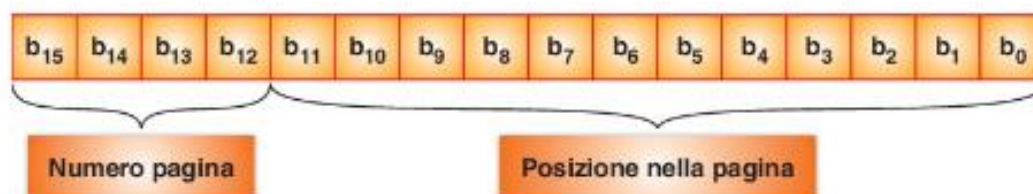
Un processo nel corso della sua evoluzione genera riferimenti alla memoria. Gli indirizzi prodotti dal processo sono *indirizzi logici* che devono essere trasformati nel corrispondente *indirizzo fisico* specificato come: numero di frame e posizione nel frame. La traduzione di un indirizzo relativo nell'indirizzo fisico viene attuata dall'hardware della MMU dividendo il campo indirizzi, di 16 bit, in due parti: i 4 bit più significativi rappresentano il numero di pagina, mentre i 12 bit a destra indicano la posizione all'interno della pagina.

Nella seguente tabella sono riportati i valori limite degli indirizzi di alcune pagine di memoria.

N. Pagina				Posizione nella Pagina												Decimale	Pagina
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	4095	
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	4096	1
0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	8191	
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	8192	2
0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	12287	
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	12288	3
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	16383	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	61440	15
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	65535	

Dall'esame della tabella si nota che i primi 4 K indirizzi, corrispondenti alla pagina 0, sono caratterizzati dall'avere i bit 12,13, 14, 15 tutti posti a 0. Analogamente gli indirizzi da 4096 a 8191, appartenenti alla pagina 1, sono caratterizzati dal fatto che i bit di ordine più elevato valgono 0001 e così via, sino agli ultimi 4 K indirizzi nei quali i bit di ordine più elevato valgono 1111.

I 16 bit di un indirizzo sono suddivisi dalla MMU in due parti per identificare il numero di pagina, tramite i 4 bit di ordine più elevato, e la posizione dentro la pagina, detta **offset** (sbilanciamento), mediante i 12 bit meno significativi.

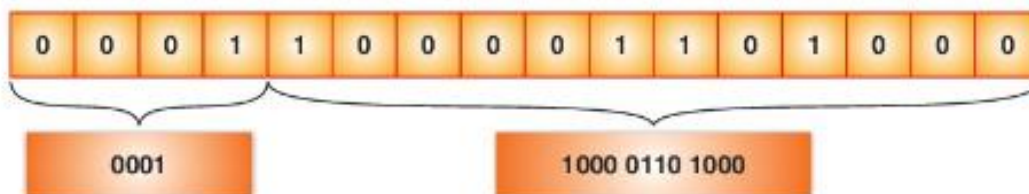


Il numero di pagina, rappresentato con 4 bit, può variare da 0 a 15, per un totale di 16 pagine, mentre la posizione nella pagina, di 12 bit, può variare da 0 a 4095, cioè per 4 K indirizzi distinti.

Per comprendere come, tramite la tabella delle pagine, avviene la trasformazione dall'indirizzo logico a quello fisico, consideriamo alcuni esempi che si riferiscono alle tabelle delle pagine dei processi P_1 , P_2 e P_3 del precedente esempio. Iniziamo con un caso semplice, calcolando l'indirizzo fisico corrispondente a 6248 del processo P_1 del quale si riporta la tabella delle pagine. Dalla tabella delle pagine si nota che ogni pagina del processo è collocata in un frame di pari valore: la pagina 0 è nel frame 0, la pagina 1 nel frame 1 e così via. Questo significa, in pratica, che indirizzo logico e indirizzo fisico coincidono. L'esercizio è utile solo per controllare che il metodo di trasformazione dia risultati corretti.

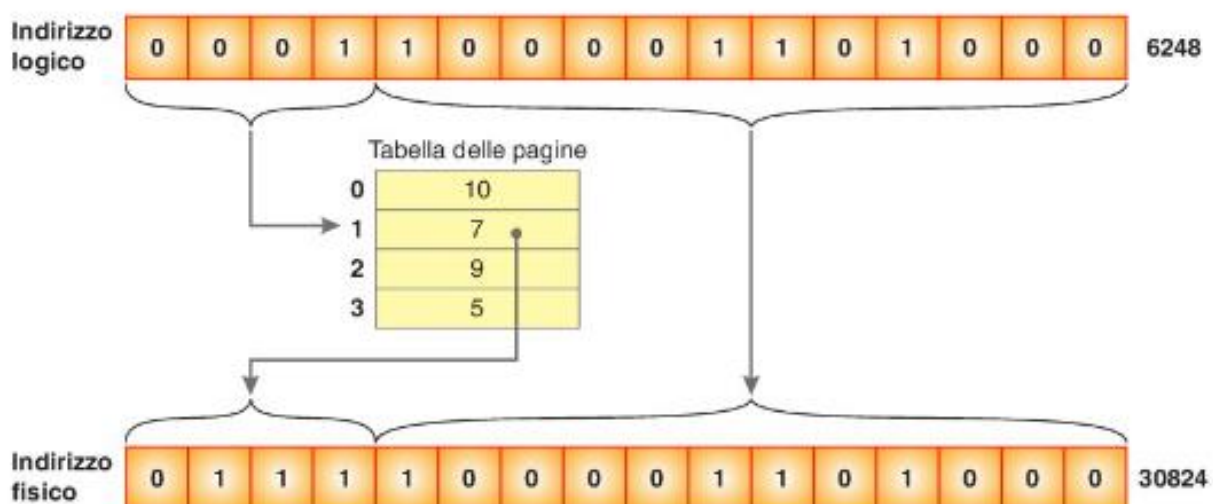
0	0
1	1
2	2
3	3

Passando dal valore decimale alla rappresentazione binaria, l'indirizzo 6248 diventa:



Poiché la pagina 1 viene collocata nel frame 1, l'indirizzo fisico si ottiene sostituendo i 4 bit del numero di pagina con i 4 bit che rappresentano il numero di frame corrispondente, mantenendo invariato il valore dell'*offset di pagina*, che diventa l'*offset del frame*. Usiamo il metodo appena sviluppato per calcolare la posizione in memoria reale dell'indirizzo logico 6248 del processo P_2 : si tratta di un indirizzo posto verso la metà della pagina 1 di P_2 . Dalla tabella delle pagine si vede che la pagina 1 di P_2 è collocata nel frame 7 della memoria e di conseguenza l'indirizzo logico 6248 si trova attorno alla metà del frame 7, quindi attorno al valore 30500.

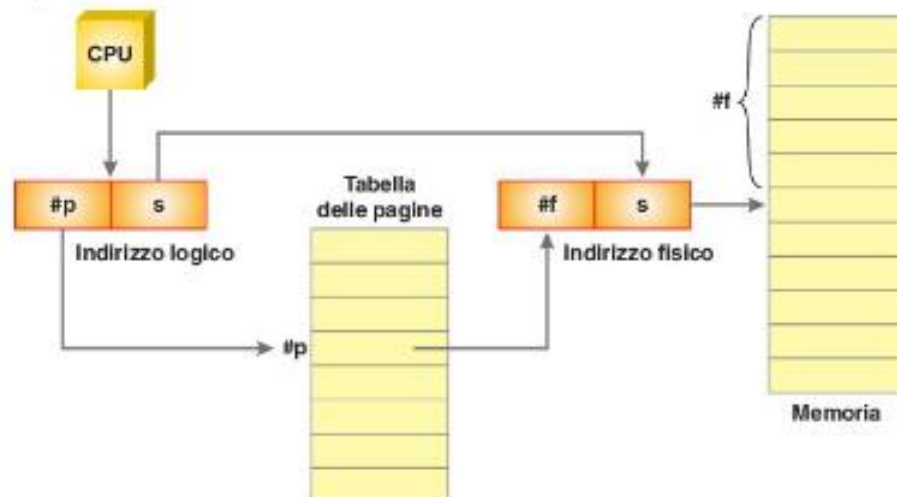
Per calcolare il nuovo indirizzo sostituiamo ai 4 bit di ordine più elevato, che rappresentano il numero di pagina (1), il valore binario del frame corrispondente (7): passando ai valori binari questo significa sostituire 0001 con i 4 bit 0111.



L'indirizzo fisico è quindi 0111 1000 0110 1000 che corrisponde al valore decimale 30824.

Le operazioni descritte nei precedenti esempi sono attuate dalla MMU che traduce gli indirizzi logici, generati da un processo in esecuzione, nei corrispondenti indirizzi fisici. Per ragioni di efficienza la traduzione è eseguita direttamente dall'hardware della MMU che usa la tabella delle pagine del processo per trasformare il numero di pagina nel corrispondente numero di frame.

Quando un nuovo processo è collocato in memoria, il sistema operativo registra nella *page table* del processo i corrispondenti *frame* di allocazione, scrive in un apposito registro della MMU la posizione in memoria della tabella delle pagine, ed esegue il processo scrivendo nel *program counter* l'indirizzo di partenza del processo.



L'esecuzione del processo ha inizio. Gli indirizzi generati dalla CPU non sono inviati direttamente alla memoria, ma alla MMU per essere trasformati.

- Nella MMU gli indirizzi, di n bit, sono scomposti in due parti:
 - gli m bit di ordine inferiore (m bit per pagine di dimensione 2^m), specificano la posizione s dell'indirizzo logico nella pagina;
 - gli $n-m$ bit rimanenti precisano il numero di pagina dell'indirizzo logico, indicato con $\#p$.
- La MMU usa il valore $\#p$ come indice per accedere alla tabella delle pagine del processo, dove è registrato il numero di frame $\#f$ di allocazione della pagina.
- La MMU costruisce l'indirizzo fisico, componendo $\#f$ con la posizione nella pagina s .
- L'indirizzo fisico è usato per accedere alla memoria.

5 La memoria virtuale

La **memoria virtuale** è il modo standard di gestire la memoria negli attuali sistemi. La memoria virtuale sfrutta il meccanismo della *paginazione*, ma non richiede la presenza in memoria del codice dell'intero processo. L'idea di fondo alla base della memoria virtuale è la seguente: non è necessario che un processo sia collocato interamente in memoria; è sufficiente che in memoria centrale sia presente la porzione di codice in esecuzione in quel momento, mentre il resto del codice del processo può rimanere sul disco, pronto per essere caricato quando è necessario.

L'inutilità di portare in memoria l'intero processo è evidenziata anche dal fatto che probabilmente certe parti del codice non saranno mai eseguite. Si pensi, per esempio, alle diverse routine per la gestione degli errori e che un'applicazione è sviluppata tenendo conto di tutte le possibili situazioni operative che si possono presentare. In una singola esecuzione, però, si verificano solo alcune di tali situazioni e il codice che gestisce gli altri casi non deve essere eseguito. Inoltre il **principio di località** garantisce che un processo, in un dato intervallo di tempo, tende ad accedere agli stessi indirizzi di memoria e quindi solo il loro contenuto deve essere presente in memoria centrale. Inoltre, avere in memoria solo una frazione dell'intero processo presenta i seguenti vantaggi.

- A parità di memoria fisica installata si possono caricare un maggior numero di processi aumentando così il grado di multiprogrammazione del sistema. All'aumentare del grado di multiprogrammazione, diminuisce la probabilità che tutti i processi siano bloccati in attesa del completamento di un'operazione di I/O e aumenta la probabilità che ci sia sempre un processo pronto per essere eseguito, migliorando l'utilizzazione della CPU.
- È possibile eseguire processi di dimensione superiore a quella delle memoria fisica. In questo modo il programmatore non ha bisogno di conoscere la dimensione della memoria sulla quale sarà eseguito il programma che sta scrivendo. La progettazione del software è semplificata e la sua **portabilità** tra sistemi è garantita.

Nella gestione della memoria virtuale ci sono due modi di gestire le parti nelle quali è suddiviso un processo:

- **memoria virtuale a pagine** quando il processo è suddiviso in blocchi della stessa dimensione;
- **memoria virtuale a segmenti** se il processo è suddiviso in parti aventi dimensione diversa.

Nel seguito viene illustrata la tecnica di gestione della memoria virtuale a pagine, mentre la memoria virtuale a segmenti è introdotta a fine paragrafo e approfondita nei *Contenuti digitali integrativi*. Con la *memoria virtuale* la dimensione di un processo è limitata dalla capacità di indirizzamento del processore. Se il processore prevede istruzioni macchina con indirizzi di n bit, è possibile indirizzare una memoria virtuale di 2^n parole. Tale memoria costituisce lo **spazio di memoria virtuale del processo** che è composto da un certo numero di pagine aventi tutte la stessa dimensione.

Lo spazio degli indirizzi virtuali è definito dal numero di bit dell'indirizzo. Considerando processori che indirizzano i byte della memoria con indirizzi di 32 bit si ha una memoria virtuale di:

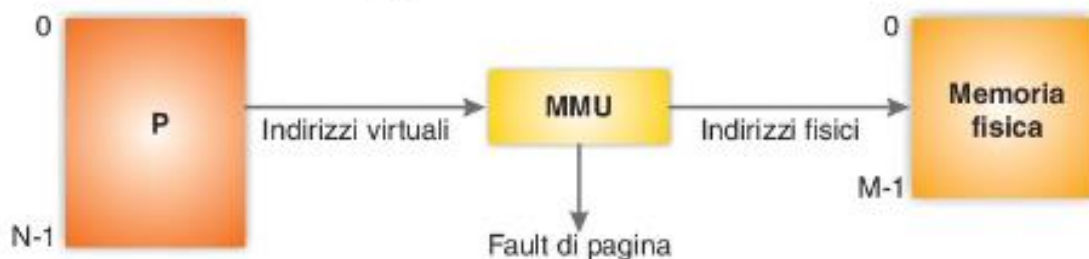
$$2^{32} \text{ byte} = 4 \cdot 2^{30} \text{ byte} = 4 \text{ GB}$$

mentre con indirizzi di memoria a 64 bit la memoria virtuale è:

$$2^{64} \text{ byte} = 2^4 \cdot 2^{20} \cdot 2^{40} \text{ byte, circa } 16.000.000 \text{ TB}$$

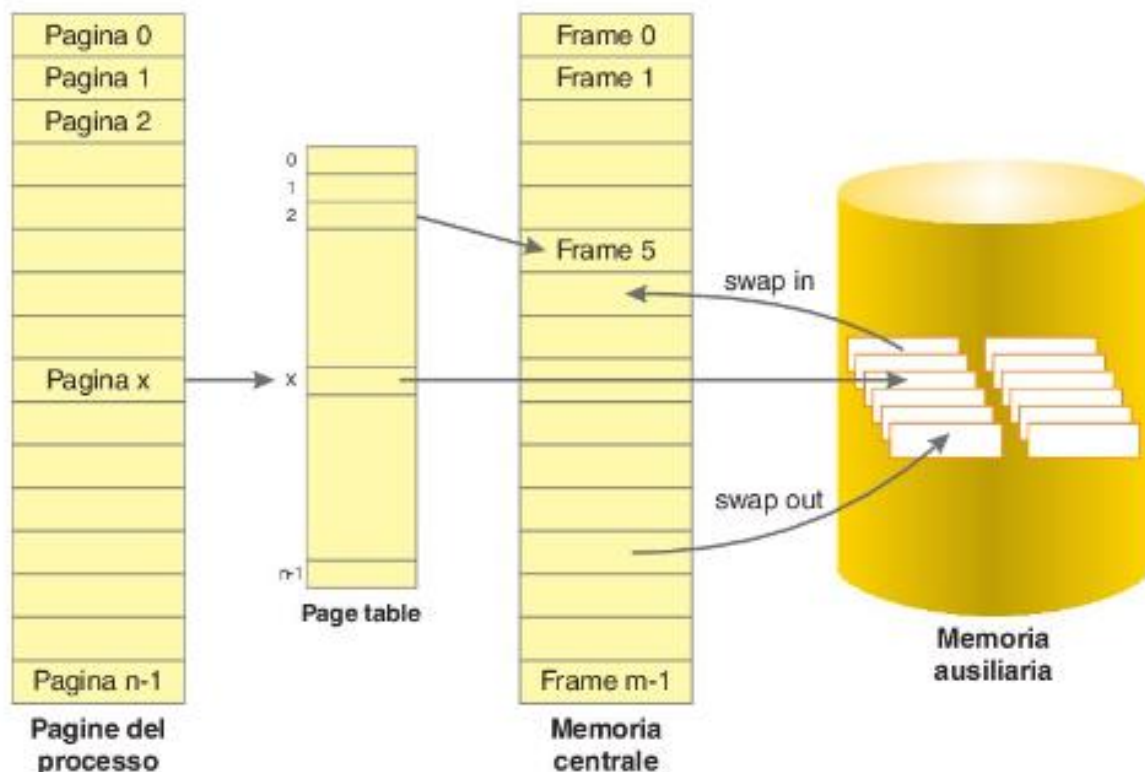
Per questa ragione si dice che, in genere, il numero di pagine è molto più grande del numero di frame di memoria reale installata.

Supponiamo che un processo abbia uno spazio di indirizzi virtuale di N parole: la memoria fisica, di M parole, costituisce lo **spazio di memoria reale** ed è composta da un certo numero di blocchi della stessa dimensione di quella delle pagine.



La MMU opera il mapping tra pagine di memoria virtuale e *frame* di memoria reale e, dato che il processo non è tutto in memoria, la MMU deve essere in grado di segnalare l'assenza in memoria di una pagina virtuale. Questa situazione si indica con il termine di **page fault** o *fault di pagina* (assenza di una pagina). Quando rileva un *page fault* la MMU invia un segnale di interruzione che provoca l'intervento del sistema operativo per caricare in memoria la pagina mancante.

La figura seguente illustra la situazione: lo spazio degli indirizzi virtuali di un processo è suddiviso in n pagine mentre lo spazio fisico in memoria centrale è suddiviso in m frame. La corrispondenza tra pagine e frame è descritta nella *tabella delle pagine*. Si noti che la tabella delle pagine deve fornire due indicazioni: la presenza o l'assenza della pagina in memoria e, in caso di pagina in memoria, in quale frame è stata collocata.



Per esempio la pagina 2 del processo si trova nel frame 5 della memoria, mentre la pagina x non è caricata in memoria. Il tentativo di accedere alla pagina x genera un *page fault*. Il sistema operativo deve allora ricopiare la pagina mancante in un frame libero eseguendo un'operazione di *swap in*.

Se non ci sono frame liberi, il sistema operativo deve liberare un frame salvando la pagina su disco con un'operazione di *swap out*. Lo *swapping* delle pagine è costoso in termini di tempo, per la necessità di operare trasferimenti da e per il disco, che richiedono diversi millisecondi per essere eseguiti.

Se il numero di pagine in memoria è troppo basso ci sono molti fault di pagina e il sistema operativo deve eseguire molti swapping di pagina con elevati costi di sistema che provocano una condizione di **thrashing**: il sistema passa il tempo a spostare pagine tra la memoria e il disco invece di eseguire istruzioni.

Le pagine di un processo in memoria in un certo momento costituiscono il **resident set** (insieme residente) del processo. In altre parole la dimensione del *resident set* è un parametro critico per il funzionamento della memoria virtuale: se il *resident set* è troppo piccolo, si ha un sovraccarico del sistema che causa il *thrashing*; se invece il *resident set* è troppo grande, ci sono in memoria molte pagine inutilizzate e il grado di multiprogrammazione diminuisce senza riduzione della percentuale di *page fault*.

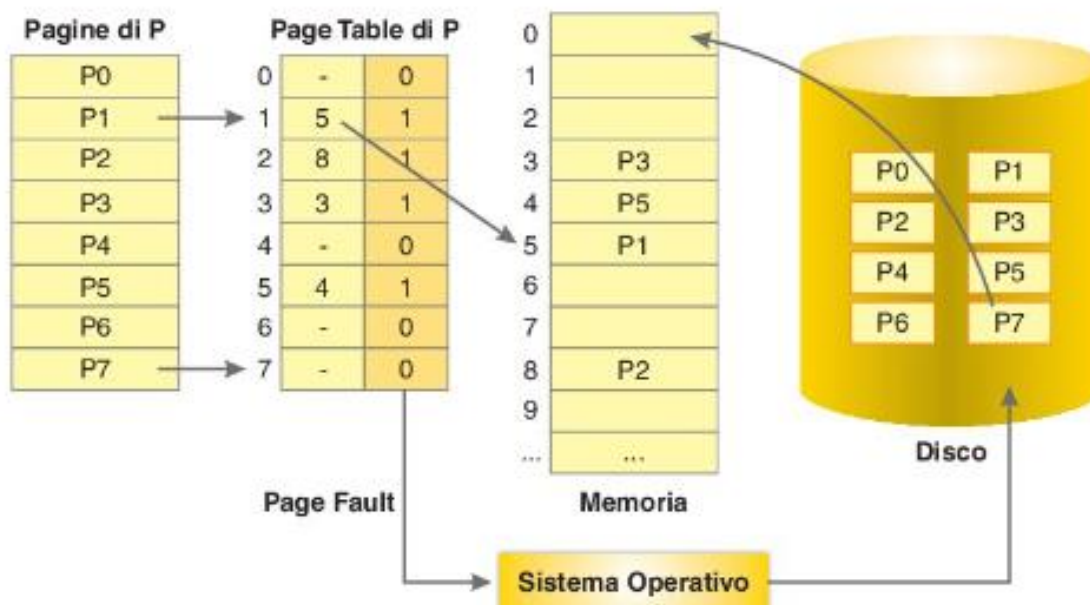
I vantaggi della memoria virtuale sono confermati dall'esperienza di molti sistemi operativi che ne fanno uso. La memoria virtuale funziona con modalità analoghe alla memoria cache, cioè valgono i **principi di località**, di cui si è già parlato nel primo capitolo:

- **Principio di località temporale**, se un programma accede a una parola di memoria è probabile che in un breve intervallo di tempo acceda di nuovo alla stessa cella di memoria.
- **Principio di località spaziale**, un programma che accede a una parola di memoria accederà probabilmente, entro breve tempo, a celle di memoria vicine.

In particolare il principio di località spaziale sta alla base della tecnica di gestione della memoria virtuale: quando un processo usa certe pagine continuerà a usarle per un certo periodo, senza accedere alle altre.

La tabella delle pagine

Vediamo ora come viene realizzata la tabella delle pagine e i compiti del sistema operativo nella gestione del *fault di pagina*. Si è già visto in precedenza che la tabella delle pagine deve informare sulla presenza o meno delle pagine in memoria. Per questa ragione in ogni riga della tabella delle pagine c'è un bit **V**, detto **bit di validità** o *bit di presente/assente*, che indica se quella riga della tabella delle pagine è valida ($V=1$), oppure non è valida ($V=0$) perché la pagina non è in memoria. Per esempio si abbia un processo **P** con 8 pagine di memoria virtuale e la corrispondente tabella delle pagine. Dall'esame della tabella delle pagine si riconosce che le pagine 1, 2, 3 e 5, per le quali si ha $V=1$, sono in memoria, mentre le rimanenti pagine non lo sono perché il bit **V** vale 0.

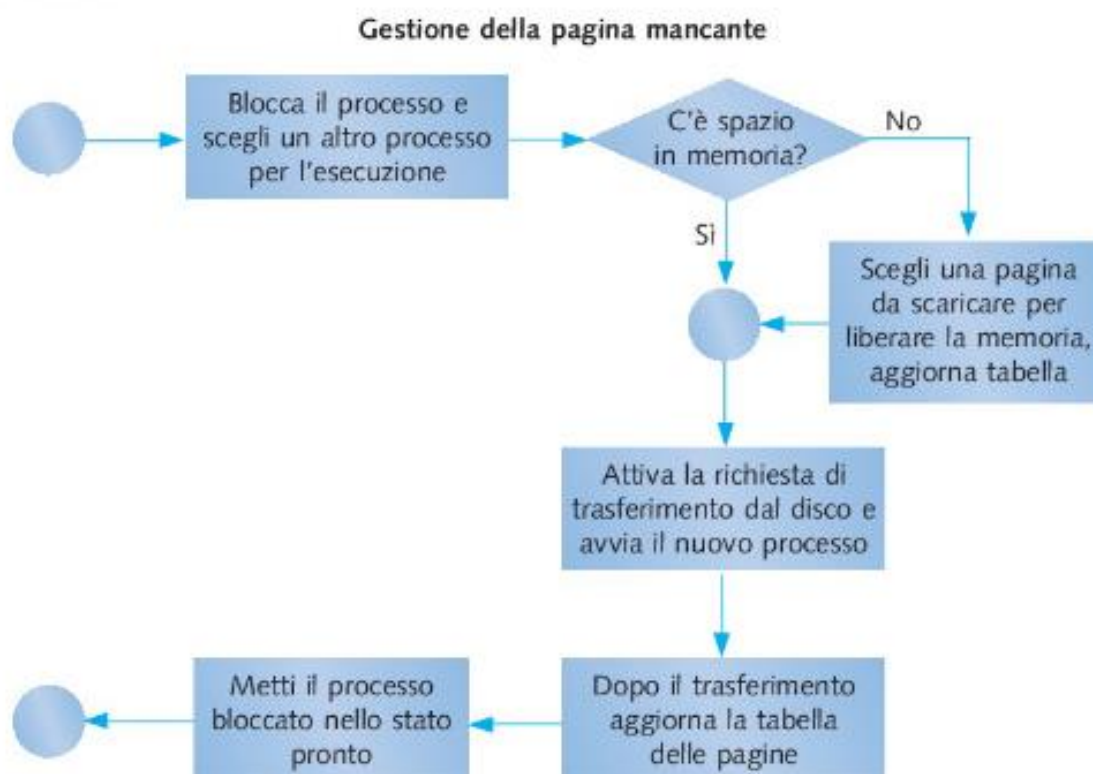


La pagina virtuale 1 è in memoria nel frame 5, mentre un tentativo di **P** di accedere alla pagina virtuale 7 genera un *fault di pagina*, perché il corrispondente bit di validità nella tabella delle pagine è 0. L'assenza della pagina in memoria causa la chiamata della routine per la gestione dell'interruzione da *page fault* e l'esecuzione delle seguenti azioni:

- Il sistema operativo mette nello stato bloccato il processo **P** che ha causato il *fault di pagina* e identifica un altro processo **Q** da eseguire, scegliendolo tra quelli nello stato *ready*.
- Il sistema operativo controlla lo stato della memoria per scegliere un frame dove collocare la pagina richiesta. Se in memoria non ci sono frame liberi, deve identificare, con un opportuno algoritmo, il frame da liberare. Per effetto della rimozione della pagina, dovrà essere aggiornata la riga della tabella delle pagine corrispondente alla pagina liberata, mettendo a 0 il bit di validità. Il frame con la pagina da rimuovere è ricopiato su disco.

- Il sistema operativo attiva il trasferimento dei dati da disco in memoria, per ricopiare la pagina richiesta nel frame prescelto. Attivato il trasferimento, il sistema operativo manda in esecuzione il processo **Q**.
- Dopo aver completato il trasferimento da disco a memoria, il sistema operativo aggiorna la tabella delle pagine del processo **P** e lo mette nello stato *ready*.

Le azioni descritte per la gestione della pagina mancante sono rappresentati dal seguente diagramma di flusso.



Dopo aver scelto la pagina da rimuovere dalla memoria, è possibile migliorare l'efficienza delle operazioni di gestione della pagina mancante evitando di ricopiare su disco la pagina in memoria: a condizione, naturalmente, che la pagina in memoria non sia stata modificata dopo il caricamento. Per questa ragione, ogni riga delle tabella delle pagine contiene spesso, oltre al bit di validità, anche un **bit di pagina modificata M**, detto anche *dirty bit*, che è impostato a 0 al momento del caricamento della pagina in memoria. Il bit **M** è in seguito messo a 1 ogni volta che il contenuto del frame viene modificato.

In questo modo il sistema operativo, prima di liberare il frame, controlla il valore di **M**: se il valore di **M** è 0, allora la pagina non è mai stata modificata e lo spazio in memoria può essere sovrascritto. In tale caso una riga della tabella delle pagine ha il formato:

V	M	Numero Frame
---	---	--------------

In generale una riga nella tabella delle pagine ha un formato del tipo:

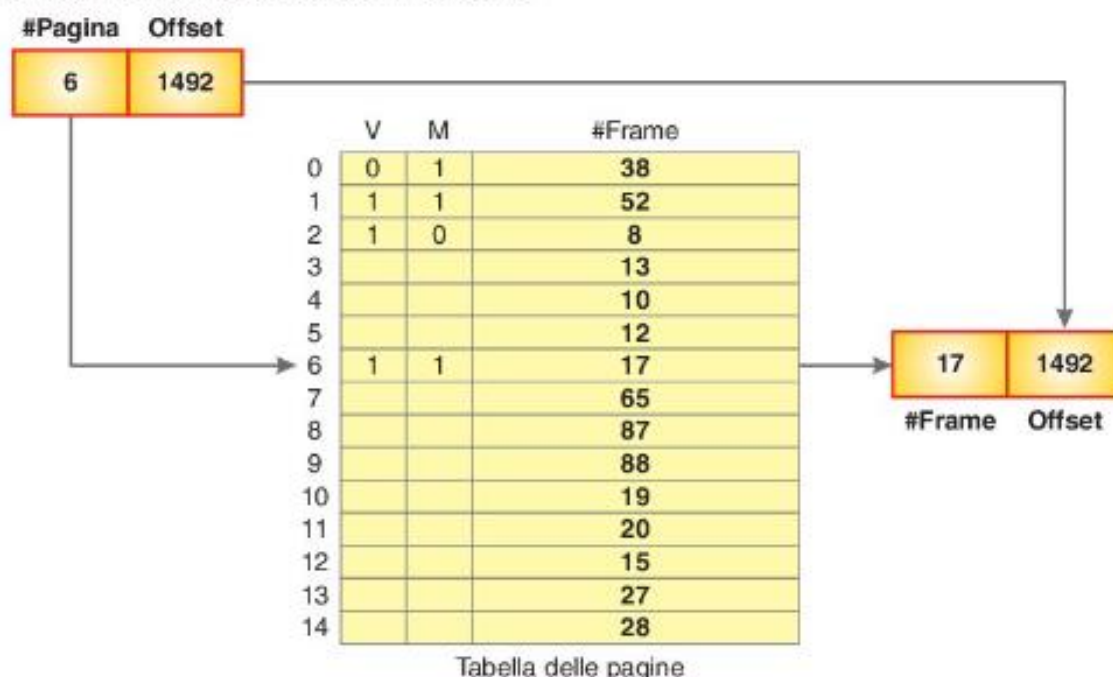
Bit di controllo	Numero Frame
------------------	--------------

Nelle varie implementazioni della memoria virtuale, tra i bit di controllo, ci sono spesso, oltre ai bit **V** ed **M**, altri flag tra i quali **U**, **R**, **W**, **X**, con il seguente significato:

- **U**, la pagina è stata usata. Questo bit in alcuni algoritmi consente la scelta della pagina da scaricare. Per esempio, il sistema operativo, in base ai valori dei bit **U** ed **M**, sceglie la pagina da eliminare secondo l'appartenenza della pagina a certe classi: prima tra le pagine che non sono mai state né usate né modificate, poi tra quelle che sono state usate ma non modificate e infine tra quelle usate e modificate.
- **R**, pagina che può essere solo letta, ma nella quale non è possibile scrivere.
- **W**, pagina nella quale è possibile scrivere.
- **X**, pagina che contiene codice ed è usata per essere eseguita.

I bit **R**, **W**, **X** sono usati per implementare meccanismi di protezione delle pagine.

L'esempio seguente descrive la trasformazione degli indirizzi virtuali in indirizzi fisici, eseguita dalla MMU usando i dati della tabella delle pagine.



Dalla tabella delle pagine si nota in particolare che:

- l'indirizzo virtuale nella posizione 1492 della pagina 6 è collocato in memoria nel frame 17 con offset 1492; la pagina 6 è stata modificata perché il bit **M** vale 1;
- la pagina 0 non è in memoria (**V**=0) e le successive informazioni, cioè il valore di **M** e il numero di frame, non hanno alcun significato;
- la pagina 1 è in memoria nel frame 52 ed è stata modificata: se il sistema operativo decide di liberare il frame 52 deve ricopiare il suo contenuto su disco;
- la pagina 2 è in memoria nel frame 8 e non è stata modificata: se il sistema operativo decide di liberare il frame 8 lo può sovrascrivere con il contenuto di un'altra pagina senza ricopiarne il valore su disco.

Nel precedente paragrafo si è osservato che, con la paginazione, aumenta il tempo di accesso alla memoria, perché, con la tabella delle pagine in memoria, ogni accesso alla memoria si traduce in due accessi, e che lo spazio riservato a un processo cresce per la presenza della tabella delle pagine. Per ridurre il tempo di accesso alla memoria, la ricerca del numero di frame corrispondente a una pagina virtuale avviene ricercandolo in una speciale memoria cache, quindi molto veloce, posta nella MMU. Questa memoria, detta **TLB** (*Translation Lookaside Buffer*), funziona come ogni altra memoria cache.

L'elemento di tabella delle pagine necessario è cercato nel buffer TLB: se si trova in TLB è immediatamente usato; diversamente, l'elemento è prelevato dalla tabella delle pagine posta in memoria e ricopiato nel TLB. Nel TLB, come avviene in tutte le memorie cache, non c'è l'intera tabella delle pagine, ma solo una sua parte. Per sapere se una riga della tabella è in TLB, senza scorrere l'intera tabella, l'accesso al TLB avviene in parallelo.

Nel caso della memoria virtuale il problema dello spazio occupato dalla tabella delle pagine non è trascurabile. Per esempio, con uno spazio di indirizzamento virtuale di 4 GB e pagine di 4 KB, la tabella delle pagine è composta da:

$$4 \text{ GB} / 4 \text{ KB} = 4 \cdot 2^{30} \text{ byte} / 4 \cdot 2^{10} \text{ byte} = 2^{20} = 1 \text{ M}$$

ovvero circa 1.000.000 di elementi per processo. Il problema dello spazio occupato dalla tabella della pagine è risolto in diversi modi: in particolare si utilizza una tecnica che prevede la possibilità di organizzare la tabella delle pagine in una struttura ad albero e di portare in memoria le sole foglie con le informazioni sulle pagine effettivamente usate dal processo.

La **segmentazione** è una tecnica di allocazione dei processi in memoria che differisce dalla paginazione perché i processi sono suddivisi in blocchi, detti **segmenti**, di differente dimensione. Analogamente a quanto succede alle pagine con la paginazione, un processo segmentato è allocato in memoria per segmenti non necessariamente adiacenti.

Con la segmentazione un processo è suddiviso in blocchi secondo criteri logici che riflettono l'organizzazione del software. Per esempio un programma potrebbe essere composto da un certo numero di segmenti secondo la strutturazione dei moduli che lo compongono: un segmento per il programma principale, uno per ogni sottoprogramma, un altro ancora per un blocco di dati e così via. La segmentazione riflette quindi la visione che il programmatore ha del proprio programma. Nel caso della paginazione, invece, un processo è formato da un unico spazio di memoria che è suddiviso artificialmente in un certo numero di blocchi.

Con la segmentazione un processo è suddiviso in blocchi, ognuno dei quali possiede una precisa identità logica e funzionale e ha un proprio spazio di indirizzamento. La segmentazione presenta, rispetto alla paginazione, i seguenti vantaggi:

- la gestione di spazi memoria condivisi è più semplice perché i dati (o il codice) da condividere possono essere inseriti in un unico segmento che viene condiviso tra i processi;
- la protezione della memoria può essere effettuata in modo puntuale secondo le esigenze e le caratteristiche funzionali del singolo segmento;
- la gestione delle strutture dati dinamiche è semplificata perché, se nell'esecuzione del processo lo spazio occupato dai dati aumenta, basta aumentare la dimensione di quello specifico segmento. Nel caso della paginazione è invece impossibile aumentare la dimensione delle pagine ed è più complicato inserire nuove pagine tra quelle assegnate al processo per allocare più spazio ai dati.

Si deve osservare che i segmenti sono allocati in memoria come avviene con i procedessi nel caso della partizioni a dimensioni variabili e, quindi, si manifestano gli stessi problemi di frammentazione esterna, come si è visto nel Paragrafo 3 nel caso dei processi. Per usare al meglio i vantaggi della segmentazione senza incorrere nei problemi della frammentazione esterna, sono stati realizzati sistemi di indirizzamento che combinano segmentazione e paginazione. In tal caso la gestione della memoria consiste nella paginazione dei segmenti che compongono un processo.